Ex no: 6	CONTAINERIZING CURRENCY CONVERTOR USING DOCKER
Date:	

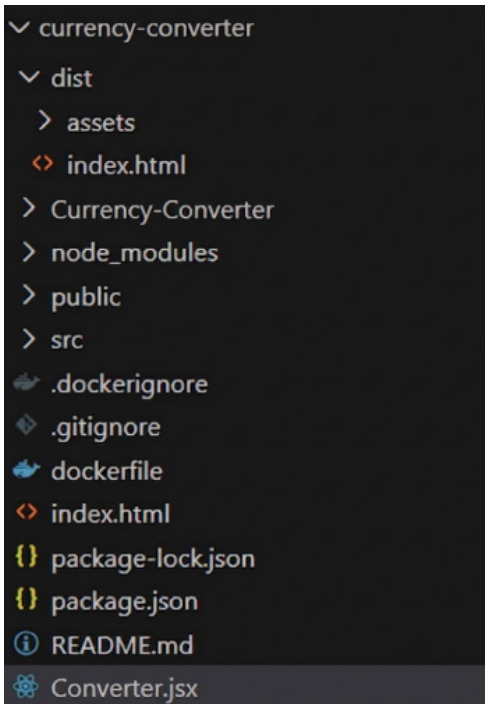
Aim:

To containerize the Currency Convertor using Docker to ensure consistent development and deployment. This involves creating a Docker image containing the React application and its dependencies, running the application inside a container, publishing the Docker image to Docker Hub, and verifying that the application can be deployed from the published image.

Requirements / Tools

1. A working Currency Convertor React project.
2. Docker Desktop installed and running.
3. Dockerfile.
4. .dockerignore file.
5. Docker Hub account.
6. Internet connection for pushing and pulling the Docker image.

Structure:



Code and Github:

```
import { useState, useEffect } from 'react'
import './App.css'
import './components/shared.css'

function App() {
  // Amount to convert
  const [amount, setAmount] = useState(1)

  // Selected base and target currencies
  const [fromCurrency, setFromCurrency] = useState('USD')
  const [toCurrency, setToCurrency] = useState('EUR')

  // Live exchange rates and available currency list fetched from API
  const [exchangeRates, setExchangeRates] = useState({})
  const [currencies, setCurrencies] = useState([])
  const [loading, setLoading] = useState(true)

  // Conversion history stored in state (resets on page refresh)
  const [history, setHistory] = useState([])

  // Fetch exchange rates whenever the base currency changes
  useEffect(() => {
    setLoading(true)
    fetch(`https://open.er-api.com/v6/latest/${fromCurrency}`)
      .then((res) => res.json())
      .then((data) => {
        setExchangeRates(data.rates || {})
        setCurrencies(Object.keys(data.rates || {}))
        setLoading(false)
      })
      .catch((err) => {
        console.error('Error fetching exchange rates:', err)
        setLoading(false)
      })
  }, [fromCurrency])

  // Calculate conversion output
  const rate = exchangeRates[toCurrency] || 0
  const convertedAmount = (amount * rate).toFixed(2)

  // Swap "From" and "To" currencies
  const handleSwap = () => {
    setFromCurrency(toCurrency)
    setToCurrency(fromCurrency)
  }

  // Saves a conversion record with unique ID
  const saveConversion = () => {
    if (!amount || amount <= 0) return

    const newRecord = {
      id: Date.now(),
      amount,
```

```
    fromCurrency,
    convertedAmount,
    toCurrency,
    rate: rate.toFixed(4),
    time: new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' }),
  }
  setHistory((prev) => [newRecord, ...prev])
}

// Removes a saved record by ID
const deleteRecord = (id) => {
  setHistory((prev) => prev.filter((item) => item.id !== id))
}

return (
  <div className="app-container">
    <header className="app-header">
      <h1> Currency Converter</h1>
      <p>Convert global currencies with real-time exchange rates.</p>
    </header>

    <div className="app-content">
      <div className="converter-card">
        {loading ? (
          <p>Loading exchange rates...</p>
        ) : (
          <>
            <div className="input-group">
              <label>Amount</label>
              <input
                type="number"
                value={amount}
                min="0"
                onChange={(e) => setAmount(Math.max(0, Number(e.target.value)))}
              />
            </div>

            <div className="currency-selection">
              <div className="input-group">
                <label>From</label>
                <select
                  value={fromCurrency}
                  onChange={(e) => setFromCurrency(e.target.value)}
                >
                  {currencies.map((curr) => (
                    <option key={curr} value={curr}>
                      {curr}
                    </option>
                  ))}
                </select>
              </div>

              <button className="swap-btn" onClick={handleSwap} title="Swap Currencies">
                ⇄
              </button>
```

```
            <div className="input-group">
              <label>To</label>
              <select
                value={toCurrency}
                onChange={(e) => setToCurrency(e.target.value)}
              >
                {currencies.map((curr) => (
                  <option key={curr} value={curr}>
                    {curr}
                  </option>
                ))}
              </select>
            </div>
          </div>

          <div className="result-section">
            <h2>
              {amount} {fromCurrency} = {convertedAmount} {toCurrency}
            </h2>
            <p className="rate-info">
              1 {fromCurrency} = {rate.toFixed(4)} {toCurrency}
            </p>
            <button className="save-btn" onClick={saveConversion}>
              Save Conversion
            </button>
          </div>
        </>
      )}
    </div>

    {history.length > 0 && (
      <div className="history-section">
        <h3>Saved Conversions ({history.length})</h3>
        <ul className="history-list">
          {history.map((item) => (
            <li key={item.id} className="history-item">
              <span>
                <strong>
                  {item.amount} {item.fromCurrency} → {item.convertedAmount} {item.toCurrency}
                </strong>
                <small> ({item.time})</small>
              </span>
              <button onClick={() => deleteRecord(item.id)}>Delete</button>
            </li>
          ))}
        </ul>
      </div>
    )}
  </div>
 </div>
)
}

export default App
```

Step 1: Open the Project

Open the terminal in the project directory.

cd Currency-Convertor

Check the project files:

The project should contain files such as:

- src
- public
- package.json
- package-lock.json
- vite.config.js
- index.html

Step 2: Create Dockerfile

Create a file named:

Dockerfile

Add the following content:

```
# Stage 1: Build React Application
FROM node:20 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Stage 2: Serve React Application using Nginx
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Step 3: Create .dockerignore

Create a file named:

.dockerignore

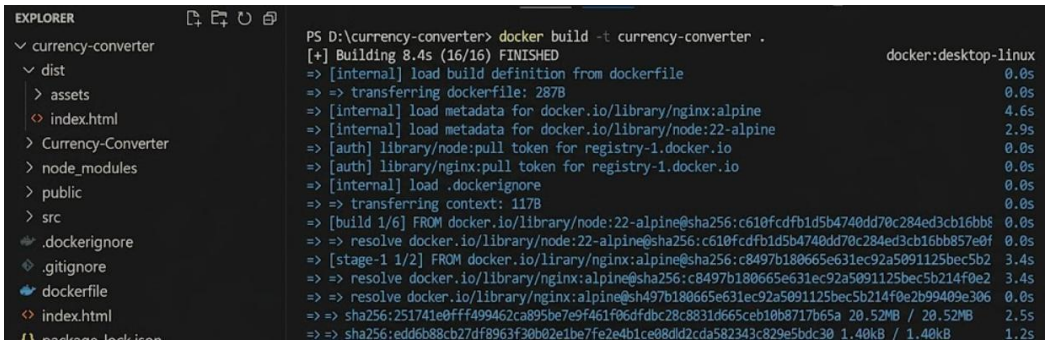
Add:

- node_modules
- dist
- .git
- .gitignore
- Dockerfile
- npm-debug.log

Step 4: Build Docker Image

Inside the project directory, execute:

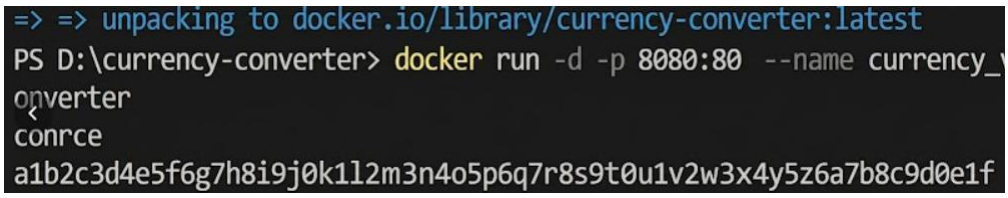
docker build -t currency-convertor .

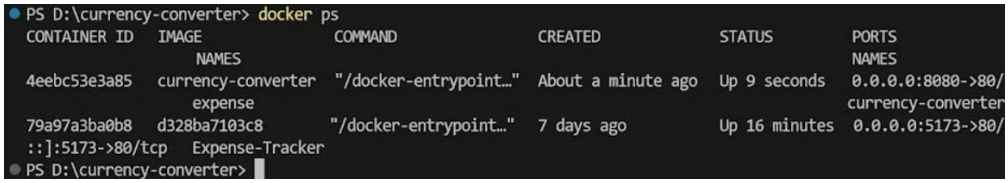


Step 5: Run Docker Container

Run the application using:

docker run -d -p 8088:5173 --name parking-app currency-convertor





Check the running containers:

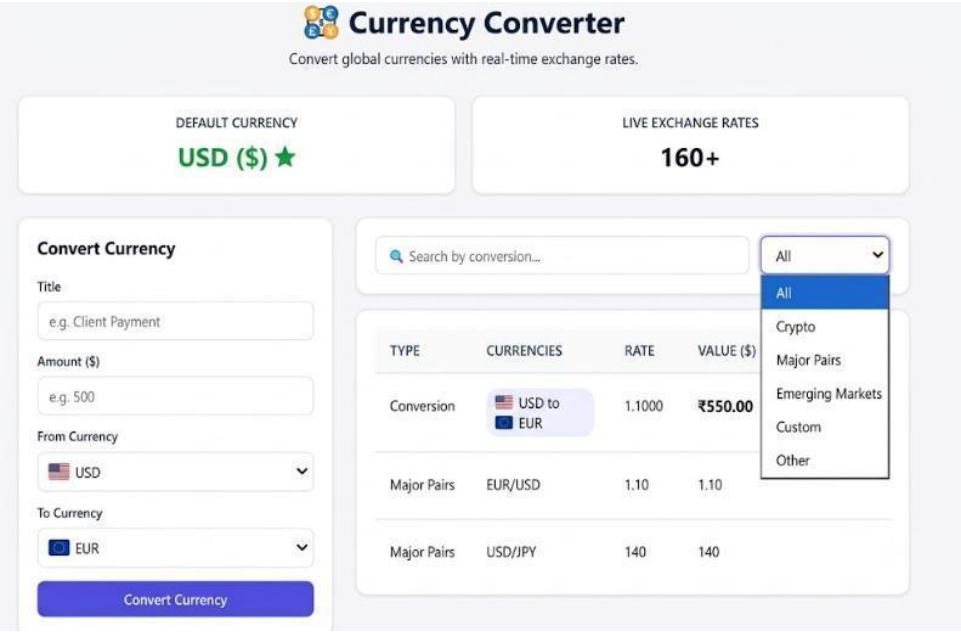
docker ps

Step 6: Verify the Application

Open a web browser and enter:

http://localhost:8080/

The expense tracker should be displayed



Step 7: Login to Docker Hub

Open the terminal and execute:

docker login

```
PS D:\currency-converter> docker login
Authenticating with existing credentials... [Username: antonnevesa10]

i Info → To login with a different account, run 'docker logout' followed by
```

Step 8 : Push docker image

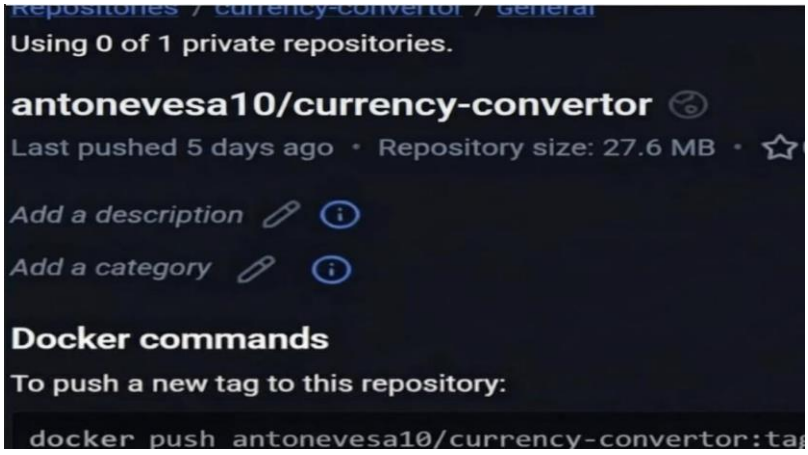
Push the Docker image:

docker push antonyrajinevesa-del/currency-convertor

```
PS D:\currency-convertor> docker push antonnevesa10/currency-convertor
The push refers to repository [docker.io/antonnevesa10/currency-conver
c8a3f9e2d1c1: Pushed
b4c9e7f8d3a2: Pushed
a7d8e9f2c3b4: Pushed
f1e2d3c4b5a6: Layer already exists
a8b7c6d5e4f: Pushed
3b7c6d5e4f3a: Pushed
7c6d5e4f3a2b: Pushed
6d5e4f3a2b1c: Pushed
5e4f3a2b1c9d: Pushed
4f3a2b1c9d8e: Layer already exists
a2b1c9d8e7f: Pushed
```

Step 9 : Verify Docker Hub Repository

Open Docker Hub and navigate to the repository:



Step 10: Stop the Container

Stop the running container:

docker stop currencyconvertr

```
PS D:\currency-convertor> docker stop currencyCONVERTOR
>> docker rm currencyCONVERTOR
currency CONVERTOR
```

Step 11: Remove the Container

Remove the stopped container:

docker rm parking-app

```
PS D:\currency-convertor> docker rmi antonevesa10/currency-convertor:latest
>> docker rmi currency-convertor:latest
Untagged: antonevesa10/currency-convertor:latest
Untagged: currency-convertor:latest
Deleted: sha256:9a91549594d84a01211156e2cd9e449698c97bf2f55a20614bd228c8bb0
```

Step 12: Pull Image from Docker Hub

Download the image from Docker Hub:

docker pull antonyrajinevesa-del/ currency-convertor:latest

```
PS D:\currency-convertor> docker pull antonevesa10/currency-convertor:latest
latest: Pulling from antonevesa10/currency-convertor
1e48613e1e68: Pull complete
44136fa355b3: Already exists
087248f71dbb: Download complete
Digest: sha256:9a91549594d84a01211156e2cd9e449698c97bf2f55a20614bd228c8bb076b5a
Status: Downloaded newer image for antonevesa10/currency-convertor:latest
docker.io/antonevesa10/currency-convertor:latest
```

Step 13: Run the Pulled Image

Create a new container using the image downloaded from Docker Hub:

docker run -d -p 8088:5173 --name expense-tracker antonyrajinevesa-del/currency-convertor:latest

Check the container:

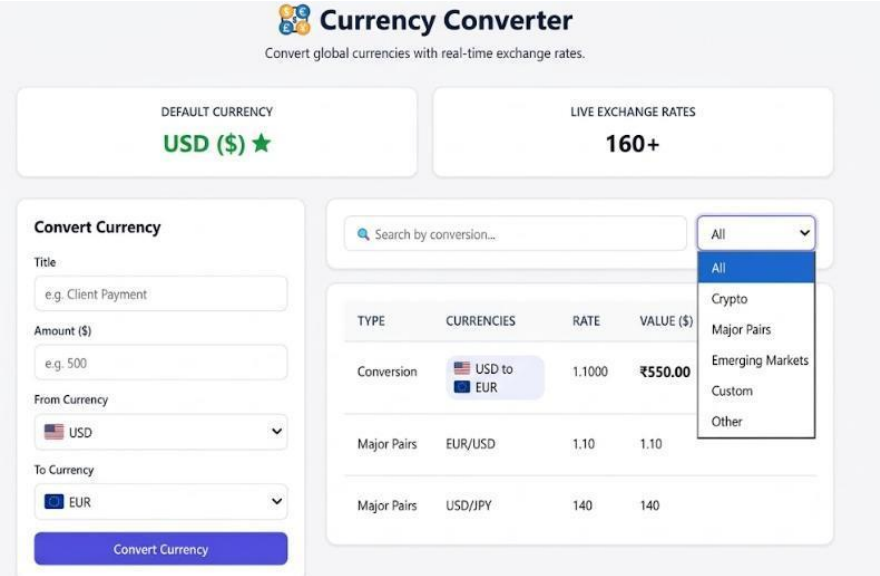
docker ps -a

Step 14: Final Verification

Open the browser:

http://localhost:8080/

Currency-Convertor should load successfully.



Rubrics:

Problem Understanding & approach (10)	Implementation and Execution (15)	Tool/ Technology usage (10)	Code Quality & Documentation (5)	Time Management (5)	Viva (5)	Total (50)

Result:

The Currency-Convertor was successfully containerized using Docker, pushed to Docker Hub, pulled, and deployed successfully.

Ex no: 7	Building and Deploying a Simple React Clothing Application using GitHub, Jenkins, and Docker.
Date:	

Aim:

To containerize a simple React clothing application using Docker to ensure consistent development and deployment environments. This involves creating a Docker image that packages the React app along with all its dependencies, enabling it to run seamlessly across different systems and platforms without compatibility issues.

Requirements:

1. Write a Dockerfile that:
    - Uses a Node.js base image to build the React app.
    - Uses Nginx as the production server to serve the built React app.
    - Exposes port 80 inside the container.
2. Create a .dockerignore file to exclude unnecessary files such as node_modules and build artifacts.
3. Show the steps to:
    - Build the Docker image.
    - Run the container so that the application is accessible on http://localhost:3000.

Structure:

```
clothing-app
├── public
│   └── images
│       ├── icons (checkmark.svg)
│       ├── products (tshirt, jeans, hoodie)
│       └── ratings (rating-40.svg, rating-45.svg)
├── src
│   ├── assets
│   ├── components
│   │   ├── Header.jsx
│   │   └── Header.css
│   ├── pages
│   │   ├── HomePage.jsx
│   │   ├── HomePage.css
│   │   ├── CheckoutPage.jsx
│   │   └── OrdersPage.jsx
│   ├── App.css
│   ├── App.jsx
│   ├── index.css
│   └── main.jsx
├── .dockerignore
├── .gitignore
├── Dockerfile
├── nginx.conf
├── eslint.config.js
├── index.html
├── package.json
└── vite.config.js
```

Code:

main.jsx:

```
import { StrictMode } from 'react'
```

```
import { createRoot } from 'react-dom/client'
import { BrowserRouter } from 'react-router';
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>,
)
```

App.jsx:

```
import { Routes, Route } from 'react-router';
import { HomePage } from './pages/HomePage';
import { CheckoutPage } from './pages/CheckoutPage';
import { OrdersPage } from './pages/OrdersPage';
import './App.css'

function App() {
  return (
    <Routes>
      <Route index element={<HomePage />} />
      <Route path="checkout" element={<CheckoutPage />} />
      <Route path="orders" element={<OrdersPage />} />
    </Routes>
  )
}

export default App
```

Header.jsx:

```
import { Link } from 'react-router';
import './Header.css';

export function Header() {
  return (
    <div className="header">
      <div className="left-section">
        <Link to="/" className="brand">StyleHub</Link>
      </div>

      <div className="middle-section">
        <input className="search-bar" type="text" placeholder="Search clothing" />
      </div>

      <div className="right-section">
        <Link className="orders-link" to="/orders">Orders</Link>
        <Link className="cart-link" to="/checkout">Cart</Link>
      </div>
    </div>
  );
}
```

HomePage.jsx:

```
import { Header } from '../components/Header';
import './HomePage.css';

const products = [
  { image: 'images/products/classic-white-tshirt.svg', name: 'Classic White Cotton T-Shirt - 2 Pack',
rating: 45, count: 87, price: '$14.90' },
  { image: 'images/products/slim-fit-blue-jeans.svg', name: 'Slim Fit Blue Denim Jeans', rating: 40,
count: 127, price: '$32.95' },
  { image: 'images/products/grey-pullover-hoodie.svg', name: 'Grey Fleece Pullover Hoodie', rating:
45, count: 56, price: '$27.99' },
];

export function HomePage() {
  return (
    <>
      <title>Clothing Store</title>
      <Header />
      <div className="home-page">
        <div className="products-grid">
          {products.map((product) => (
            <div className="product-container" key={product.name}>
              <div className="product-image-container">
                <img className="product-image" src={product.image} />
              </div>

              <div className="product-name limit-text-to-2-lines">
                {product.name}
              </div>

              <div className="product-rating-container">
                <img className="product-rating-stars" src={`images/ratings/rating-
${product.rating}.svg`} />
                <div className="product-rating-count link-primary">
                  {product.count}
                </div>
              </div>

              <div className="product-price">
                {product.price}
              </div>

              <div className="product-quantity-container">
                <select>
                  <option value="1">1</option>
                  <option value="2">2</option>
                  <option value="3">3</option>
                  <option value="4">4</option>
                  <option value="5">5</option>
                  <option value="6">6</option>
                  <option value="7">7</option>
                  <option value="8">8</option>
                  <option value="9">9</option>
                  <option value="10">10</option>
```

```
            </select>
           </div>

           <div className="product-spacer"></div>

           <div className="added-to-cart">
            <img src="images/icons/checkmark.svg" />
            Added
           </div>

           <button className="add-to-cart-button button-primary">
            Add to Cart
           </button>
          </div>
        ))}
       </div>
     </div>
    </>
  );
}
```

OrdersPage.jsx:

```
import { Header } from '../components/Header';

export function OrdersPage() {
  return (
    <>
      <title>Orders</title>
      <Header />
      <div className="page-message">You have no orders yet.</div>
    </>
  );
}
```

.dockerignore:

```
node_modules
dist
.git
.gitignore
.dockerignore
Dockerfile
npm-debug.log
```

Dockerfile:

```
# Stage 1: Build React app
FROM node:20-alpine AS build
WORKDIR /app
COPY package.json package-lock.json* ./
RUN npm install
COPY . .
RUN npm run build

# Stage 2: Serve with Nginx
FROM nginx:stable-alpine
```

```
# Copy Nginx config (SPA routing)
COPY nginx.conf /etc/nginx/conf.d/default.conf

# Copy React build files to Nginx (Vite outputs to /dist)
COPY --from=build /app/dist /usr/share/nginx/html

# Expose port 80
EXPOSE 80

# Run Nginx
CMD ["nginx", "-g", "daemon off;"]
```

nginx.conf:

```
server {
  listen 80;
  server_name _;

  root /usr/share/nginx/html;
  index index.html;

  # React Router (BrowserRouter) support: send unknown paths to index.html
  location / {
    try_files $uri /index.html;
  }
}
```

package.json:

```
{
  "name": "clothing-store",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite --host --port 3000",
    "build": "vite build",
    "preview": "vite preview --host --port 3000",
    "lint": "eslint ."
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-router": "^7.1.0"
  },
  "devDependencies": {
    "@eslint/js": "^9.17.0",
    "@vitejs/plugin-react": "^4.3.4",
    "eslint": "^9.17.0",
    "eslint-plugin-react-hooks": "^5.0.0",
    "eslint-plugin-react-refresh": "^0.4.16",
    "globals": "^15.14.0",
    "vite": "^5.4.11"
  }
}
```

Steps:

1. Open terminal in the clothing-app project folder.
2. Build Docker image.

```
docker build -t clothing-store .
```

3. Run container (host port 3000 mapped to container port 80).

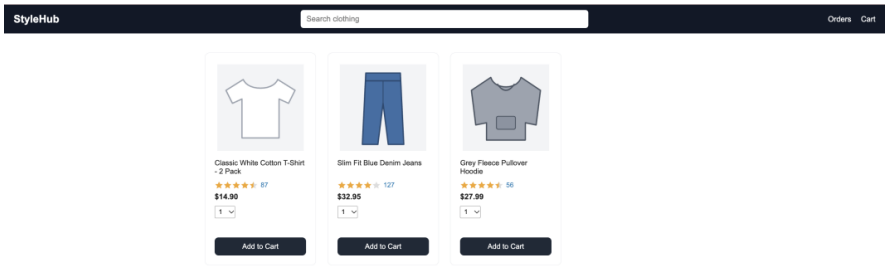
```
docker run -d -p 3000:80 --name clothing-app clothing-store
```

4. Open a browser: http://localhost:3000

Output:

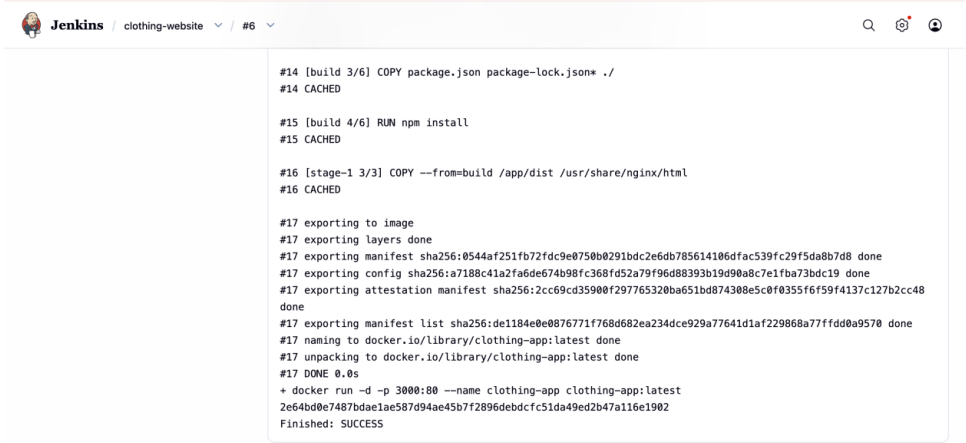
```
nevesaaxd@Nevesas-MacBook-Air clothing-app % docker build -t cl
othing-app .
[+] Building 26.4s (17/17) FINISHED     docker:desktop-linux
 => [internal] load build definition from Dockerfile    0.0s
 => => transferring dockerfile: 514B                    0.0s
 => [internal] load metadata for docker.io/library/ngi  4.4s
 => [internal] load metadata for docker.io/library/nod  4.4s
 => [auth] library/nginx:pull token for registry-1.doc  0.0s
 => [auth] library/node:pull token for registry-1.dock  0.0s
 => [internal] load .dockerignore                       0.0s
 => => transferring context: 113B                       0.0s
 => [stage-1 1/3] FROM docker.io/library/nginx:stable  11.6s
 => => resolve docker.io/library/nginx:stable-alpine@s  0.0s
 => => sha256:de314aae67cdc0a41219dd 19.87MB / 19.87MB  8.6s
```

```
nevesaaxd@Nevesas-MacBook-Air clothing-app % docker run -d -p 3000:80 --name clothing-app clothing-app:latest
3eb152c221b6ed45437f3c82fb1de7f7577473bb8cfadfc5482e7592782c4330
nevesaaxd@Nevesas-MacBook-Air clothing-app %
```



Step 1: Create a New Freestyle Project in Jenkins

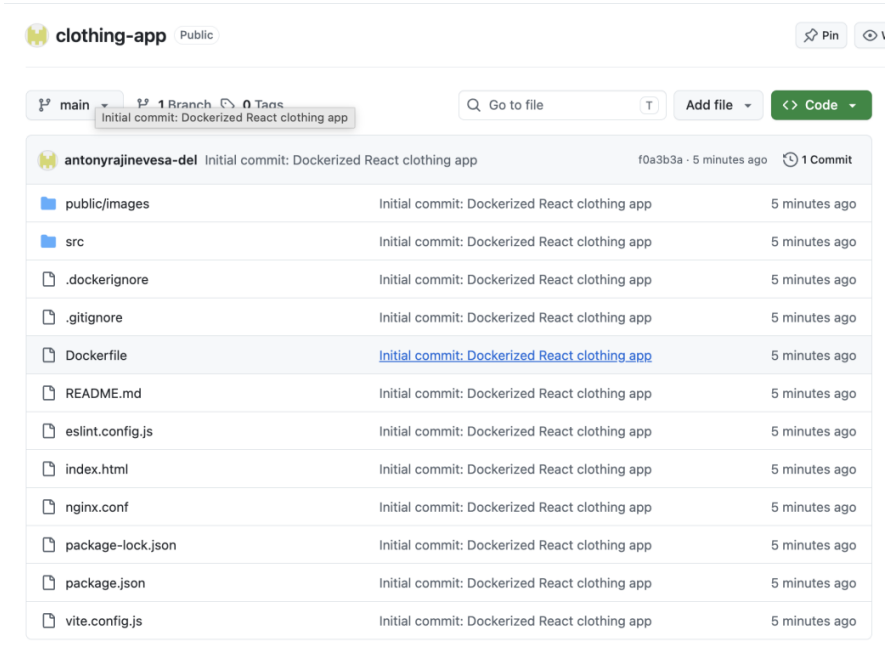
- Open Jenkins Dashboard
- Click on “New Item”
- Enter a project name (e.g., clothing-website)
- Select “Freestyle project”
- Click “OK” to proceed

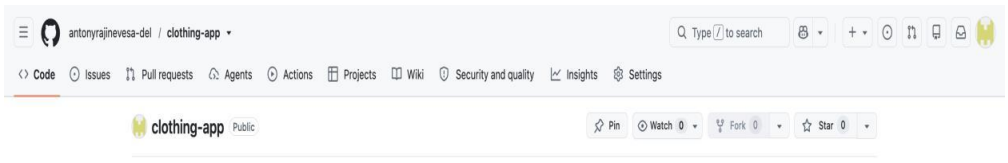


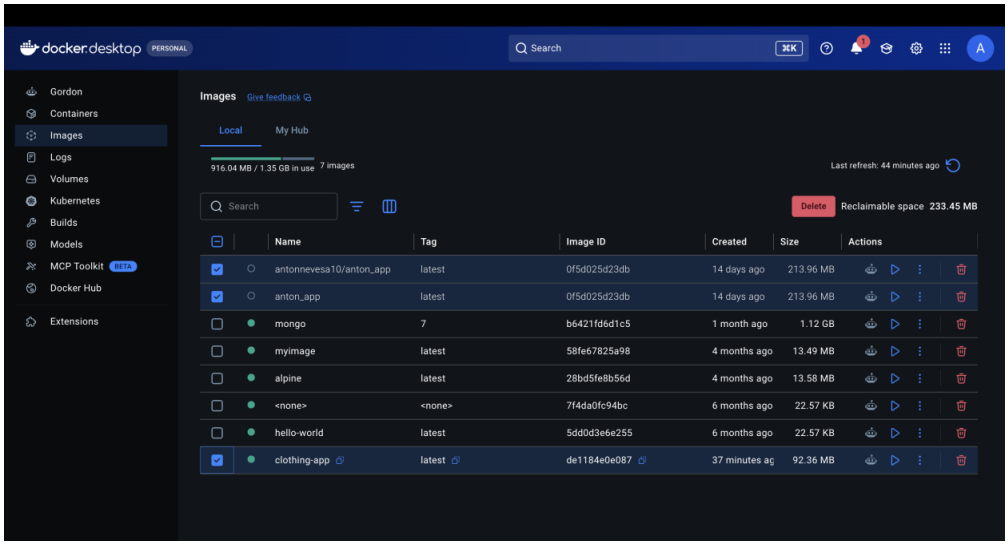
Step 2: Create a New Repository in GitHub

Step 3: Initialize Git Repository, Make First Commit, and Push to GitHub

```
git init
git add .
git commit -m "Initial commit: clothing store"
git branch -M main
git remote add origin https://github.com/<your-username>/Clothing-Store.git
git push -u origin main
```



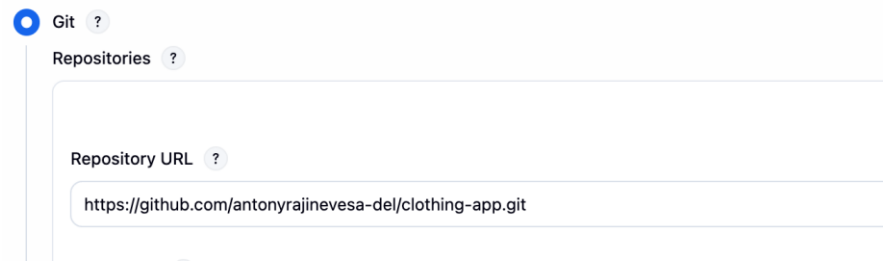




Step 4: Connect GitHub Repository to Jenkins

In the job configuration, under Source Code Management, select Git and paste the repository URL:

`https://github.com/<your-username>/Clothing-Store.git`



Step 5: Specify Branch to Build (Change master to main)

```
*/main
```

Step 6: Configure Docker Build and Run Commands in Jenkins

Under Build Steps, add "Execute Windows batch command" and enter:

```
REM Stop old container if running
docker stop clothing-app 2>nul
docker rm clothing-app 2>nul

REM Build Docker image
docker build -t clothing-store:latest .

REM Run container (host port 3000 -> container port 80)
docker run -d -p 3000:80 --name clothing-app clothing-store:latest
```

Step 7: Verify Build Execution and Git Integration Details

Click "Build Now" and open the build's Console Output. The build should finish with "Finished: SUCCESS".

```
#17 exporting to image
#17 exporting layers done
#17 exporting manifest sha256:0544af251fb72fdc9e0750b0291bdc2e6db785614106dfac539fc29f5da8b7d8 done
#17 exporting config sha256:a7188c41a2fa6de674b98fc368fd52a79f96d88393b19d90a8c7e1fba73bdc19 done
#17 exporting attestation manifest sha256:2cc69cd35900f297765320ba651bd874308e5c0f0355f6f59f4137c127b2cc48 done
#17 exporting manifest list sha256:de1184e0e0876771f768d682ea234dce929a77641d1af229868a77ffdd0a9570 done
#17 naming to docker.io/library/clothing-app:latest done
#17 unpacking to docker.io/library/clothing-app:latest done
#17 DONE 0.0s
+ docker run -d -p 3000:80 --name clothing-app clothing-app:latest
2e64bd0e7487bdae1ae587d94ae45b7f2896debdcfc51da49ed2b47a116e1902
Finished: SUCCESS
```

Step 8: Install Project Dependencies and Serve React App Locally (optional

- npm install
- npm run build
- npm install -g serve
- serve -s dist -l 3000

```
Port 3000 is in use, trying another one...

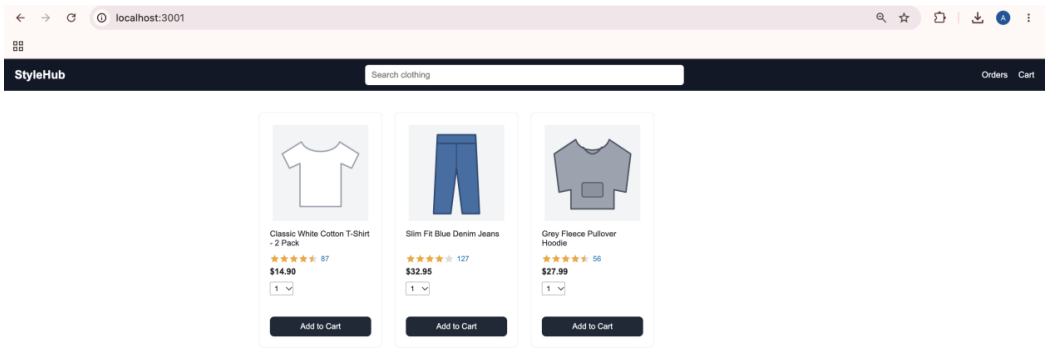
  VITE v5.4.21  ready in 74 ms

  ➜  Local:   http://localhost:3001/
  ➜  Network: http://192.0.0.2:3001/
  ➜  press h + enter to show help
```

OUTPUT:

Verify Deployment

- Open a browser and go to: http://localhost:3000 (if Jenkins host = your PC)



Rubrics:

Problem Understanding & approach (10)	Implementation and Execution (15)	Tool/ Technology usage (10)	Code Quality & Documentation (5)	Time Management (5)	Viva (5)	Total (50)

Result:

The React clothing application was successfully containerized using Docker. It can now be built, deployed, and executed consistently across all platforms with minimal setup effort, ensuring a reliable and reproducible development and deployment process.

Ex no: 8	Creating a Server Instance in AWS
Date:	

Aim

To create and configure a virtual server using Amazon EC2 (Elastic Compute Cloud) in the AWS Management Console.

Procedure

Step 1: Log in to AWS

1. Navigate to https://aws.amazon.com/console.
2. Sign in using your AWS credentials.
3. From the AWS Management Console, open the EC2 Dashboard.

Step 2: Launch a New Instance

1. Click Launch Instance.
2. Enter a name for your instance (e.g., MyServerInstance).
3. Choose an Amazon Machine Image (AMI):
   - Select Amazon Linux 2 AMI (Free Tier eligible).
4. Choose an Instance Type:
   - Select t2.micro (Free Tier eligible).
5. Under Key Pair (login):
   - Create a new key pair (e.g., my-key-pair).
   - Download the .pem file and store it securely.
6. Under Network Settings:
   - Create a new security group.
   - Allow SSH (port 22) for remote access.
   - Optionally allow HTTP (port 80) if you plan to host a web application.
7. Leave storage and advanced settings as default.
8. Click Launch Instance.

Step 3: View Instance Status

1. Go to EC2 Dashboard → Instances → Running Instances.
2. Wait for the Instance State to show Running.
3. Note down the Public IPv4 address — this will be used to connect to your server.

Step 4: Connect to the Instance

1. Select your instance → Click Connect.
2. Choose the SSH Client tab and copy the SSH command shown.
3. Open your terminal and navigate to the directory where your key file is saved.

Change the file permissions of the key pair:

```
chmod 400 your-key.pem
```

4. Connect to the Instance

Connect to your Amazon Linux 2 instance using the SSH command:

```
ssh -i "your-key.pem" ec2-user@<EC2-Public-IP>
```

(Replace <EC2-Public-IP> with your actual instance's public IPv4 address.)

5. Once connected, you'll see a terminal prompt similar to the one below:

```
[ec2-user@ip-172-31-45-102 ~]$
```

6. This indicates a successful SSH connection to your AWS EC2 instance running Amazon Linux 2.

Step 5: Verify Server Operation

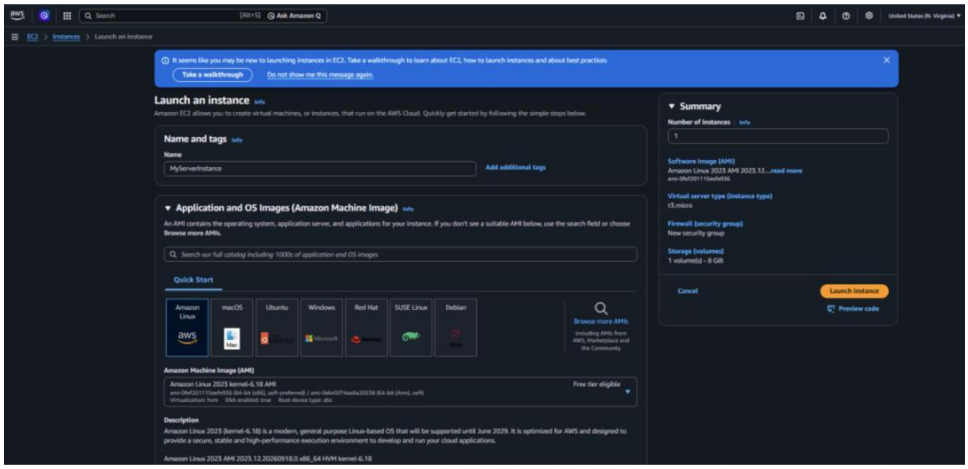
7. Run a basic Linux command to confirm the system information and kernel version:

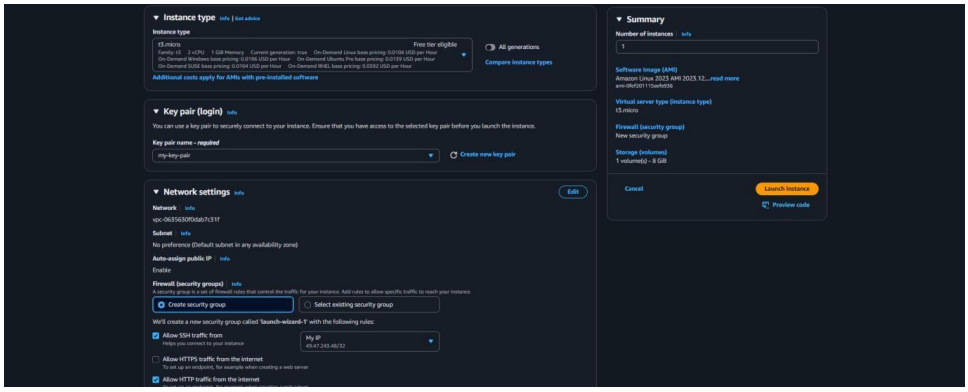
```
uname -a
```

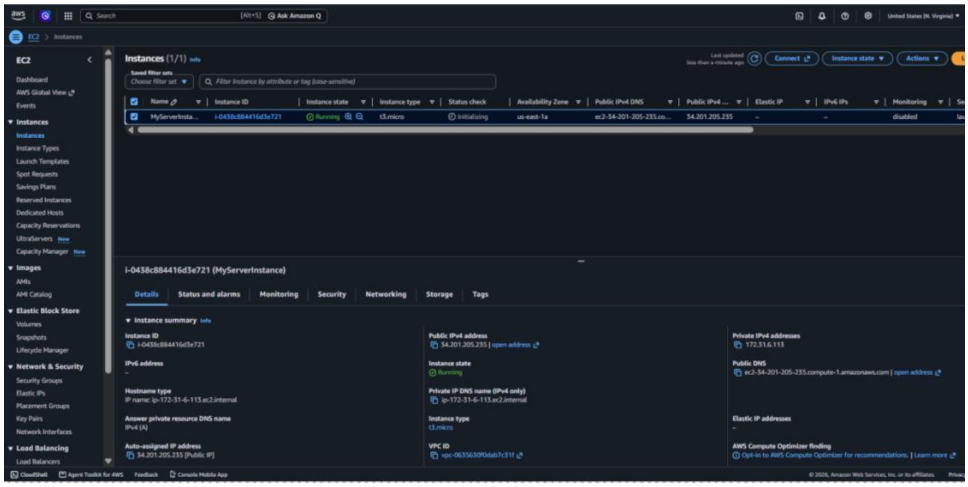
Output (as viewed on Amazon Linux 2)

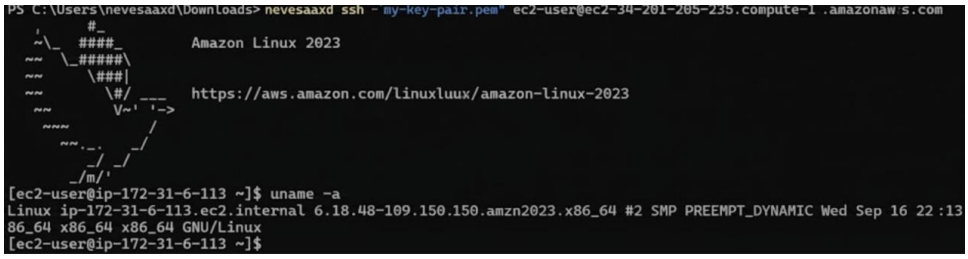
```
Linux ip-172-31-45-102 5.10.210-208.788.amzn2.x86_64 #1 SMP Tue Mar 12
18:43:13 UTC 2024 x86_64 x86_64 x86_64 GNU/Linux
```

Output









Rubrics

Problem Understanding & approach (10)	Implementation and Execution (15)	Tool/ Technology usage (10)	Code Quality & Documentation (5)	Time Management (5)	Viva (5)	Total (50)

Result

A server was successfully created on AWS using EC2. The instance was launched, configured, and accessed via SSH, confirming that the EC2 instance is operational as a cloud server.

Ex no: 9	Running Docker on AWS EC2 to Pull and Execute a "Hello World" Image
Date:	

Aim

To create and configure an AWS EC2 instance, install Docker, and verify the Docker setup by pulling and running the official "hello-world" image from Docker Hub.

Procedure

Step 1: Launch an EC2 Instance

1. Log in to AWS Management Console → Go to EC2 Dashboard.
2. Click Launch Instance.
3. Choose Ubuntu Server 22.04 LTS or Amazon Linux 2 as the AMI.
4. Select an instance type (e.g., t2.micro – Free Tier eligible).
5. Create or choose an existing key pair for SSH access.
6. Configure Security Group:
   - Allow SSH (port 22) for remote login.
7. Click Launch Instance.

Step 2: Connect to the Instance

From your terminal, connect using SSH:

```
ssh -i "your-key.pem" ubuntu@<EC2-Public-IP>
```

(Replace <EC2-Public-IP> with your instance's public IPv4 address.)

Step 3: Update the System

For Amazon Linux:

```
sudo yum update -y
```

Step 4: Install Docker

For Amazon Linux 2:

```
sudo amazon-linux-extras install docker -y
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker ec2-user
```

Logout and log back in for group permissions to take effect.

Step 5: Verify Docker Installation

```
docker --version
```

Step 6: Pull the Hello World Image

```
docker pull hello-world
```

Step 7: Run the Container

```
docker run hello-world
```

Output

```
[ec2-user@ip-172-31-6-113 ~]$ sudo yum update -y
Amazon Linux 2023 Kernel Livepatch repository                                    686 kB/s |  82 kB     00:00
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-6-113 ~]$ sudo amazon-linux-extras install docker -y
sudo: amazon-linux-extras: command not found
[ec2-user@ip-172-31-6-113 ~]$ sudo dnf install docker -y
Last metadata expiration check: 0:01:03 ago on Mon Sep 28 14:49:39 2026.
Dependencies resolved.
================================================================================================================
 Package                    Architecture       Version                          Repository               Size
================================================================================================================
Installing:
 docker                     x86_64             25.0.16-1.amzn2023.0.4           amazonlinux              46 M
Installing dependencies:
 container-selinux          noarch             4:2.245.0-1.amzn2023             amazonlinux              58 k
 containerd                 x86_64             2.2.7-1.amzn2023.0.1             amazonlinux              25 M
 iptables-libs              x86_64             1.8.8-3.amzn2023.0.2             amazonlinux             401 k
 iptables-nft               x86_64             1.8.8-3.amzn2023.0.2             amazonlinux             183 k
 libcgroup                  x86_64             3.0-1.amzn2023.0.1               amazonlinux              75 k
 libnetfilter_conntrack     x86_64             1.0.8-2.amzn2023.0.2             amazonlinux              58 k
 libnfnetlink               x86_64             1.0.1-19.amzn2023.0.2            amazonlinux              30 k
 libnftnl                   x86_64             1.2.2-2.amzn2023.0.2             amazonlinux              84 k
 pigz                       x86_64             2.5-1.amzn2023.0.4               amazonlinux              83 k
 runc                       x86_64             1.3.6-1.amzn2023.0.1             amazonlinux             4.0 M

Transaction Summary
```

```
Complete!
[ec2-user@ip-172-31-6-113 ~]$ sudo systemctl start docker
[ec2-user@ip-172-31-6-113 ~]$ sudo systemctl enable docker
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /usr/lib/systemd/system/docker.service.
[ec2-user@ip-172-31-6-113 ~]$ docker --version
Docker version 25.0.14, build 0bab007
[ec2-user@ip-172-31-6-113 ~]$
```

```
[ec2-user@ip-172-31-6-113 ~]$ docker --version
Docker version 25.0.14, build 0bab007
[ec2-user@ip-172-31-6-113 ~]$ sudo docker pull hello-world
Using default tag: latest
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest
docker.io/library/hello-world:latest
[ec2-user@ip-172-31-6-113 ~]$ sudo docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

```
[ec2-user@ip-172-31-6-113 ~]$ sudo docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
hello-world   latest    e2ac70e7319a   6 months ago   10.1kB
[ec2-user@ip-172-31-6-113 ~]$ sudo docker ps -a
CONTAINER ID   IMAGE         COMMAND    CREATED          STATUS                      PORTS     NAMES
30ed450d05dd   hello-world   "/hello"   49 seconds ago   Exited (0) 48 seconds ago             zealous_lamport
[ec2-user@ip-172-31-6-113 ~]$
```

```
[ec2-user@ip-172-31-6-113 ~]$ sudo docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
hello-world   latest    e2ac70e7319a   6 months ago   10.1kB
[ec2-user@ip-172-31-6-113 ~]$
```

Rubrics

Problem Understanding & approach (10)	Implementation and Execution (15)	Tool/ Technology usage (10)	Code Quality & Documentation (5)	Time Management (5)	Viva (5)	Total (50)

Result

Docker was successfully installed on the AWS EC2 instance, and the “hello-world” container ran successfully, confirming that the Docker setup and image pull from Docker Hub worked as expected.

Ex no: 10	Building, Running and Publishing a Full Stack Application in Docker Hub
Date:	

Aim

To containerize and deploy a full stack web application (React frontend + Node.js backend) using Docker, publish the Docker images to Docker Hub, run the containers, and publish the source code to GitHub.

Application Used

Student Management System – a minimal full stack application with a React frontend and Node.js + Express backend. The application supports adding, viewing, and deleting student records.

Requirements

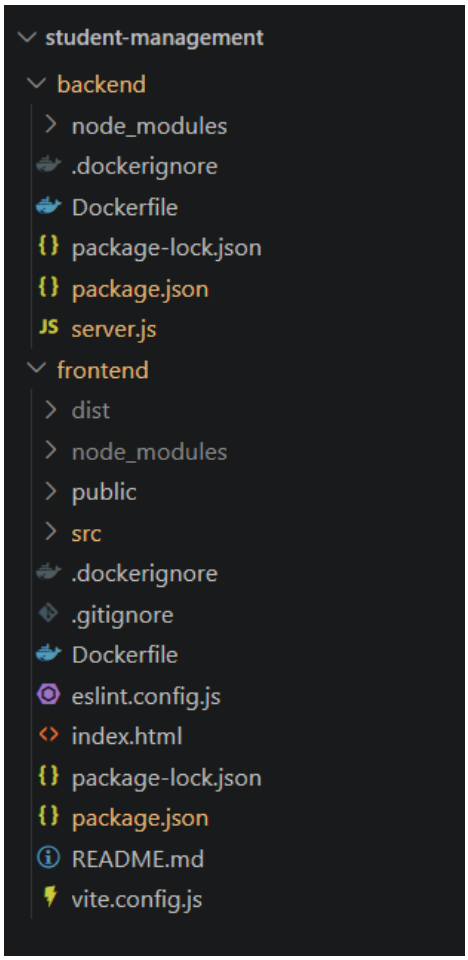
Simple full stack application (React frontend + Node.js backend)

Docker installed

Docker Hub account

GitHub account

Structure



Code

1. Frontend – src/App.jsx

```
import { useEffect, useState } from "react";
import "./App.css";

const API_URL = "http://localhost:4000/api/students";

function App() {
  const [students, setStudents] = useState([]);
  const [name, setName] = useState("");
  const [department, setDepartment] = useState("");
  const [year, setYear] = useState("");
  const [email, setEmail] = useState("");

  const fetchStudents = async () => {
    const response = await fetch(API_URL);
    const data = await response.json();
    setStudents(data);
  };

  useEffect(() => {
    fetchStudents();
  }, []);

  const addStudent = async (e) => {
    e.preventDefault();

    if (!name || !department || !year || !email) {
      alert("Please fill all fields");
      return;
    }

    await fetch(API_URL, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({
        name,
        department,
        year,
        email,
      }),
    });

    setName("");
    setDepartment("");
    setYear("");
    setEmail("");

    fetchStudents();
  };
```

```
const deleteStudent = async (id) => {
  await fetch(`${API_URL}/${id}`, {
    method: "DELETE",
  });

  fetchStudents();
};

return (
  <div className="container">
    <h1>Student Management System</h1>

    <form onSubmit={addStudent} className="student-form">
      <input
        type="text"
        placeholder="Student Name"
        value={name}
        onChange={(e) => setName(e.target.value)}
      />

      <input
        type="text"
        placeholder="Department"
        value={department}
        onChange={(e) => setDepartment(e.target.value)}
      />

      <input
        type="text"
        placeholder="Year"
        value={year}
        onChange={(e) => setYear(e.target.value)}
      />

      <input
        type="email"
        placeholder="Email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
      />

      <button type="submit">Add Student</button>
    </form>

    <h2>Student List</h2>

    {students.length === 0 ? (
      <p>No students found.</p>
    ) : (
      <div className="student-list">
        {students.map((student) => (
          <div className="student-card" key={student.id}>
            <h3>{student.name}</h3>
            <p>Department: {student.department}</p>
            <p>Year: {student.year}</p>
            <p>Email: {student.email}</p>
```

```
            <button onClick={() => deleteStudent(student.id)}>
              Delete
            </button>
          </div>
        ))}
      </div>
    )}
  </div>
 );
}

export default App;
```

2. Frontend – src/main.jsx

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
 <StrictMode>
  <App />
 </StrictMode>
);
```

3. Backend – server.js

```
const express = require("express");
const cors = require("cors");

const app = express();
const PORT = 4000;

app.use(cors());
app.use(express.json());

let students = [];

// Get all students
app.get("/api/students", (req, res) => {
 res.json(students);
});

// Add student
app.post("/api/students", (req, res) => {
 const student = {
  id: Date.now(),
  name: req.body.name,
  department: req.body.department,
  year: req.body.year,
  email: req.body.email,
 };

 students.push(student);
```

```
  res.status(201).json(student);
});

// Delete student
app.delete("/api/students/:id", (req, res) => {
  const id = Number(req.params.id);

  students = students.filter((student) => student.id !== id);

  res.json({
    message: "Student deleted successfully",
  });
});

// Home route
app.get("/", (req, res) => {
  res.send("Student Management Backend is Running!");
});

app.listen(PORT, () => {
  console.log(`Backend running on http://localhost:${PORT}`);
});
```

Procedure

Step 1: Project Setup

Create two folders: frontend for the React application and backend for the Node.js Express server.

```
student-management/
├── frontend/   → React application
└── backend/    → Node.js Express server
```

The frontend provides the Student Management user interface. The backend provides the REST API for student data.

Step 2: Create Dockerfiles

Frontend Dockerfile:

```
# Build Stage
FROM node:20 AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Production Stage
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Backend Dockerfile:

```
FROM node:20
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
```

```
EXPOSE 4000
CMD ["node", "server.js"]
```

Step 3: Create .dockerignore

Create a .dockerignore file in both frontend and backend folders.

```
node_modules
npm-debug.log
.dockerignore
```

Step 4: Build Docker Images

Build the backend Docker image:

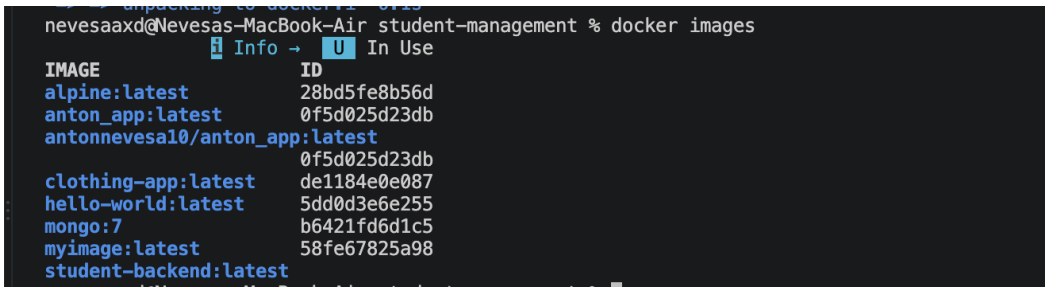
docker build -t student-backend ./backend

Build the frontend Docker image:

docker build -t student-frontend ./frontend

Verify the images:

docker images



Docker images and containers verification.

Step 5: Run Containers Locally

Run the backend container:

docker run -d -p 4000:4000 --name student-backend-container student-backend

Run the frontend container:

docker run -d -p 3000:80 --name student-frontend-container student-frontend

Check running containers:

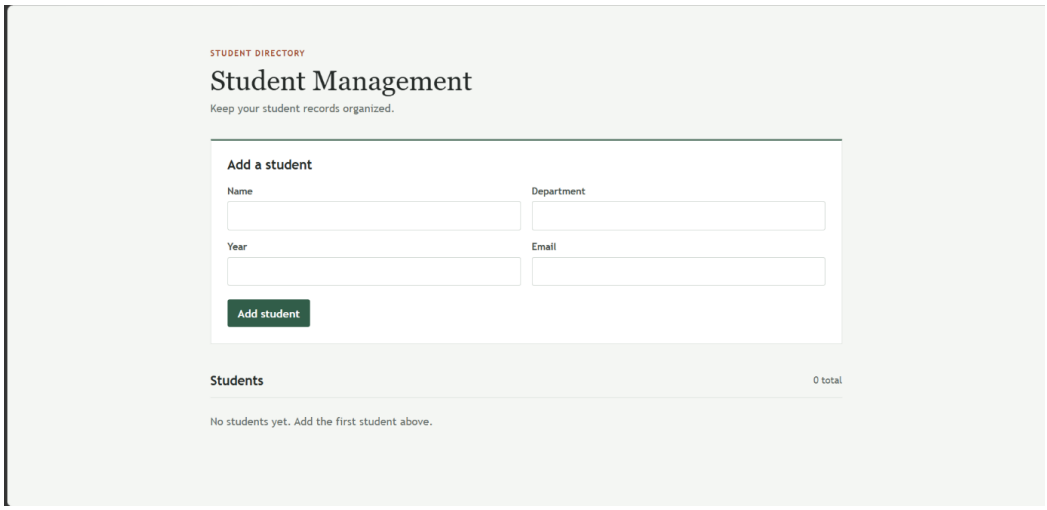
docker ps

Open the application in a browser:

http://localhost:3000

Backend endpoint:

http://localhost:4000



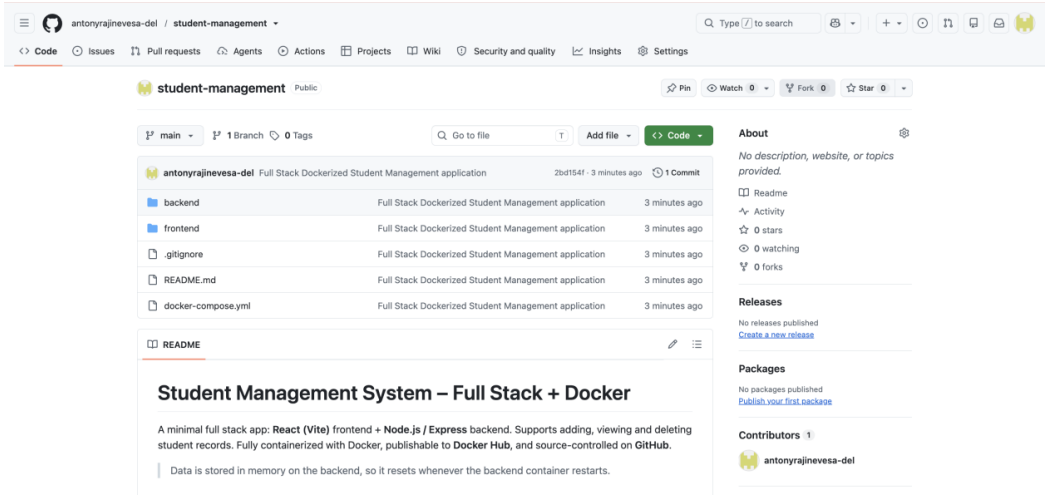
Student Management System running in the browser.

Step 6: Push Project to GitHub

Initialize the Git repository and push the project to GitHub.

```
git init
git add .
git commit -m "Full Stack Dockerized Student Management application"
git branch -M main
git remote add origin https://github.com/<username>/<repo-name>.git
git push -u origin main
```

Verify the GitHub repository and confirm that the frontend and backend source code and Dockerfiles are visible.



Step 7: Push Images to Docker Hub

Login to Docker Hub:

```
docker login
```

Tag and push the backend image:

```
docker tag student-backend <username>/student-backend:v1
docker push <username>/student-backend:v1
```

Tag and push the frontend image:

```
docker tag student-frontend <username>/student-frontend:v1
docker push <username>/student-frontend:v1
```

```
nevesaaxd@Nevesas-MacBook-Air student-management % docker login
Authenticating with existing credentials... [Username: antonnevesa10]

i Info → To login with a different account, run 'docker logout' followed by 'docker login'


Login Succeeded
```

Docker Hub image publishing.

```
nevesaaxd@Nevesas-MacBook-Air student-management % docker push antonnevesa10/student-backend:v1
The push refers to repository [docker.io/antonnevesa10/student-backend]
0bb7bd28fbdf: Layer already exists
30de66f8fdaf: Pushed
3bf6338910b6: Already exists
44136fa355b3: Already exists
599d07e8492b: Pushing [==================================>            ]  49.28MB/64.48MB
5f77b2a79d55: Pushing [================>                              ]  70.25MB/203.1MB
444403cccfb4: Layer already exists
8f963a2bbd53: Pushed
78dea6468cd9: Layer already exists
6260d7010302: Pushed
a1f2e003d826: Pushed
a2c2ba26a655: Pushed
badfca60eea0: Pushed
2127e02b1b4e: Pushed
```

Step 8: Testing and Validation

The published images can be pulled on another machine or VM and run to confirm that the application works consistently.

```
docker pull <username>/student-frontend:v1
docker pull <username>/student-backend:v1
```

Run the pulled images and verify the application from the browser.

```
nevesaaxd@Nevesas-MacBook-Air student-management % docker pull antonnevesa10/student-frontend:v1
v1: Pulling from antonnevesa10/student-frontend
Digest: sha256:061f32707fd9cb0e4401438d9248b04c9949cf9892ee11385033adb21126985c
Status: Image is up to date for antonnevesa10/student-frontend:v1
docker.io/antonnevesa10/student-frontend:v1
nevesaaxd@Nevesas-MacBook-Air student-management %
```

Pulling and running the published Docker images.

```
nevesaaxd@Nevesas-MacBook-Air student-management % docker tag student-frontend antonnevesa10/student-frontend:v1
nevesaaxd@Nevesas-MacBook-Air student-management % docker push antonnevesa10/student-frontend:v1
The push refers to repository [docker.io/antonnevesa10/student-frontend]
94e7f9dfc058: Pushed
44136fa355b3: Mounted from antonnevesa10/student-backend
858f76146c12: Pushed
a9986cd6f37d: Pushed
766d3311d5df: Pushed
ccea4a153c59: Pushed
943c389642fa: Pushed
bc862a9910f5: Pushed
d766d06974fa: Pushed
1efec2485f45: Pushed
092a00bbc0ec: Pushed
v1: digest: sha256:061f32707fd9cb0e4401438d9248b04c9949cf9892ee11385033adb21126985c size: 856
```

Pulling and running the published Docker images.

Rubrics

Problem Understanding & approach (10)	Implementation and Execution (15)	Tool/ Technology usage (10)	Code Quality & Documentation (5)	Time Management (5)	Viva (5)	Total (50)

Result

The Student Management full stack application was successfully containerized using Docker. The React frontend and Node.js backend were built as Docker images, the application was run using Docker containers, the images were published to Docker Hub, and the source code was prepared for GitHub.